

Warehouse Inventory Drone — Complete Code Reference

A line-by-line explanation of every file in the project, in the order the files were built.

Table of Contents

1. Background concepts you need first
 2. File build order and dependencies
 3. `build_islands.py` — shelf models
 4. `build_boxes.py` — boxes and QR textures
 5. `build_scanner_drone.py` — the vehicle model
 6. `build_v2_world.py` — assembling the world
 7. `4022_gz_x500_scanner` — PX4 airframe
 8. `flow_flight_test.py` — GPS-free flight test
 9. `waypoint_nav.py` — navigation test
 10. `warehouse_scanner.py` — the main system
 11. Python syntax reference
 12. Debugging methodology
-

1. Background Concepts You Need First

1.1 What Gazebo Is and How It Reads Models

Gazebo is a physics simulator. It does not know about your objects until you describe them in **SDF** (Simulation Description Format), which is XML.

A model lives in a folder containing two files:

```
island_1/
  model.config  what this model is called
  model.sdf     what it looks like and how it collides
```

Gazebo searches for these folders in the directories listed in the `GZ_SIM_RESOURCE_PATH` environment variable. PX4 adds its own model directory to that path automatically, which is why we write our models into `~/PX4-Autopilot/Tools/simulation/gz/models/`.

1.2 The Coordinate System

Gazebo uses **ENU**: East-North-Up.

```
      Z (up)
      |
      |
+----- Y (north / forward)
/
/
X (east / right)
```

All distances are in **metres**. All rotations are in **radians** (90 degrees = 1.5708 radians = $\pi/2$).

PX4, however, uses **NED**: North-East-Down. This mismatch matters and is handled explicitly in the flight code:

```
Gazebo X  ->  NED East
Gazebo Y  ->  NED North
Gazebo Z  ->  NED Down, with the sign flipped
```

1.3 The Pose Tag

Every object in SDF is placed with a `<pose>` :

```
<pose>X Y Z ROLL PITCH YAW</pose>
```

Six numbers: three for position in metres, three for rotation in radians.

Critical detail: Gazebo positions a box by its **centre**, not by a corner. A 2 m tall box whose centre is at Z=1.0 has its base exactly on the ground. This single fact causes most of the placement mistakes made during this project.

1.4 Visual vs Collision

```
<visual>    what the camera and renderer see
<collision> what the physics engine uses for contact
```

These are separate on purpose. A production model might use a detailed mesh for the visual and a simple box for collision, because collision checking against a detailed mesh is expensive. In this project both use the same box geometry.

1.5 Link and Static

A `<link>` is a rigid body. Parts within one link cannot move relative to each other. If two parts need to move relative to each other you use two links joined by a `<joint>`.

All the legs and shelves of an island are welded together, so one link is enough.

```
<static>true</static>
```

This tells the physics engine the object never moves and is not affected by gravity. It is both correct for a warehouse shelf and much cheaper to simulate.

1.6 Why We Generate XML With Python

Writing one shelf by hand is about 100 lines of XML. Three shelves is 300 lines. Fifty-four boxes would be several thousand. Generating the XML from a Python script means:

- The geometry is written once as a formula, not repeated by hand
- Changing a dimension means changing one constant, not editing 54 files
- Mistakes are made once, not 54 times

The script does not create 3D objects. It creates **text files** which Gazebo then reads.

2. File Build Order and Dependencies

<code>build_islands.py</code>	creates <code>island_1</code> , <code>island_2</code> , <code>island_3</code> models
<code>build_boxes.py</code>	creates <code>invbox_001</code> .. <code>invbox_054</code> models also writes <code>inventory_ground_truth.json</code>
<code>build_scanner_drone.py</code>	creates the <code>x500_scanner</code> vehicle model
	v
<code>build_v2_world.py</code>	reads <code>inventory_ground_truth.json</code> , writes <code>warehouse_v2.sdf</code> and <code>marker_map.json</code>
	v
<code>4022_gz_x500_scanner</code>	PX4 airframe file, placed in the PX4 source tree
	v
<code>flow_flight_test.py</code>	verifies GPS-free flight works at all
<code>waypoint_nav.py</code>	verifies route following works
<code>warehouse_scanner.py</code>	the full system

The three `build_*` model scripts are independent of each other and can be run in any order. `build_v2_world.py` must run after all three because it references the models they create.

3. `build_islands.py`

3.1 Purpose

Generates three shelf island models. Each island is a double-sided rack, 12 m long, 1.6 m deep, 2 m tall, with three shelf levels supported by eight legs.

3.2 Imports

```
import os
```

`os` is Python's interface to the operating system. Used here for creating directories and building file paths. It ships with Python, no installation needed.

3.3 The Model Directory

```
GZ_MODELS = os.path.expanduser('~/.PX4-Autopilot/Tools/simulation/gz/models')
```

`os.path` is a submodule of `os` dealing only with filesystem paths.

`expanduser()` replaces the `~` shortcut with the real home directory:

```
'~/PX4-Autopilot' -> '/home/furk/PX4-Autopilot'
```

Python does not expand `~` automatically. Without this call the path would be treated as a literal folder named `~` and would not be found.

Writing `~` rather than `/home/furk` keeps the script working on any machine, regardless of username.

The variable name is uppercase by convention. In Python, `UPPERCASE` signals a constant that should not be modified while the program runs.

3.4 Dimension Constants

```
ISLAND_DEPTH = 1.6      # X axis, the thin side
ISLAND_LENGTH = 12.0    # Y axis, the long side
ISLAND_HEIGHT = 2.0     # Z axis
LEG_SIZE = 0.08
PLATE_THICKNESS = 0.04
```

Why these values:

Constant	Value	Reasoning
ISLAND_DEPTH	1.6 m	Double-sided rack: 0.8 m usable depth per face
ISLAND_LENGTH	12.0 m	Warehouse is 20 m long, leaving 4 m clearance each end
ISLAND_HEIGHT	2.0 m	Fits three levels, low enough to fly over
LEG_SIZE	0.08 m	Realistic steel upright thickness
PLATE_THICKNESS	0.04 m	Realistic shelf board thickness

3.5 Shelf Levels

```
LEVELS_Z = [0.35, 1.0, 1.65]
```

A Python **list**: an ordered collection in square brackets. Elements are accessed by index starting at zero.

```
LEVELS_Z[0]    # 0.35
LEVELS_Z[2]    # 1.65
len(LEVELS_Z)  # 3
```

The name means "the Z coordinates of the shelf levels".

Three levels spread evenly over 2 m:

```
2.00  top of frame
      0.35 gap
1.65  ----- level 3
      0.65 gap    (a 0.3 m box fits with clearance)
1.00  ----- level 2
      0.65 gap
0.35  ----- level 1
      0.35 gap
0.00  floor
```

3.6 Island Positions

```
islands_x = [-6.2, -1.6, 3.0]
```

The X coordinate of the centre of each island. Derived from the warehouse width:

```
Warehouse spans X = -10 to +10, so 20 m total
Three islands at 1.6 m each           = 4.8 m
Remaining space for four aisles       = 15.2 m
```

Laid out left to right:

```
-10.0  wall
      aisle 1
-6.2   ISLAND 1 centre    islands_x[0]
      aisle 2
-1.6   ISLAND 2 centre    islands_x[1]
      aisle 3
 3.0   ISLAND 3 centre    islands_x[2]
      aisle 4
10.0   wall
```

These are centres, not edges, because that is how Gazebo positions objects.

3.7 Leg Positions Along the Island

```
leg_y_positions = [-6, -2, 2, 6]
```

Four rows of legs along the 12 m length, spaced 4 m apart. Two legs at the ends alone would look structurally implausible.

```
Y:  -6  -----  -2  -----  +2  -----  +6
      leg         leg         leg         leg
```

3.8 Leg Positions Across the Island

```
leg_x_offsets = [-ISLAND_DEPTH/2 + LEG_SIZE/2,
                  ISLAND_DEPTH/2 - LEG_SIZE/2]
```

Evaluating:

```
ISLAND_DEPTH/2 = 0.8    distance from centre to edge
LEG_SIZE/2      = 0.04   half the leg thickness

left  leg centre = -0.8 + 0.04 = -0.76
right leg centre =  0.8 - 0.04 =  0.76
```

The `LEG_SIZE/2` correction exists because Gazebo positions by centre. Placing a leg centre exactly at -0.8 would put half of it outside the island:

```
leg centre at -0.80  -> leg spans -0.84 to -0.76 (overhangs)
leg centre at -0.76  -> leg spans -0.80 to -0.72 (flush)
```

This is the same reasoning that will appear again when placing boxes on shelves and when working out flight altitudes.

3.9 The Main Loop

```
for idx, cx in enumerate(islands_x):
```

`enumerate()` yields both the index and the value on each iteration:

```
for idx, cx in enumerate([-6.2, -1.6, 3.0]):
    # idx=0, cx=-6.2
    # idx=1, cx=-1.6
    # idx=2, cx=3.0
```

`idx` is needed to number the model names. `cx` is printed for confirmation but is not used in the model file itself, because a model always describes its geometry relative to its own origin. Where the island actually sits in the world is decided later, in `build_v2_world.py`.

This separation is worth internalising:

```
model.sdf    the shape of the object, centred on its own origin
world.sdf    where each object is placed in the world
```

3.10 Creating the Model Folder

```
model_name = f"island_{idx+1}"
model_dir = os.path.join(GZ_MODELS, model_name)
os.makedirs(model_dir, exist_ok=True)
```

An **f-string** embeds variables in text. Prefix the string with `f` and put expressions in braces:

```
idx = 0
f"island_{idx+1}"    # "island_1"
```

`idx+1` because list indices start at zero but human-readable names should start at one.

`os.path.join()` joins path components with the correct separator for the operating system. Concatenating with `+` and a hard-coded `/` would break on Windows.

`os.makedirs()` creates the directory. `exist_ok=True` means "do not raise an error if it already exists", which allows the script to be run repeatedly.

3.11 The Config File

```
config = f'''<?xml version="1.0"?>
<model>
  <name>{model_name}</name>
  <version>1.0</version>
  <sdf version="1.9">model.sdf</sdf>
</model>'''
```

Triple quotes `'''` allow a string to span multiple lines.

The XML declares the model's name and points to the file containing the geometry. Without `model.config`, Gazebo does not recognise the folder as a model at all.

XML tag syntax:

```
<tag>content</tag>
    opening    closing, marked with a slash
```

3.12 Generating the Legs

```
links = []
for ly in leg_y_positions:
    for lx in leg_x_offsets:
        links.append(f'''
<visual name="leg_{ly}_{lx}_visual">
  <pose>{lx} {ly} {ISLAND_HEIGHT/2} 0 0 0</pose>
  <geometry><box><size>{LEG_SIZE} {LEG_SIZE} {ISLAND_HEIGHT}</size></box></geometry>
  <material>
    <ambient>0.3 0.2 0.1 1</ambient>
    <diffuse>0.35 0.22 0.12 1</diffuse>
  </material>
</visual>
<collision name="leg_{ly}_{lx}_collision">
  <pose>{lx} {ly} {ISLAND_HEIGHT/2} 0 0 0</pose>
  <geometry><box><size>{LEG_SIZE} {LEG_SIZE} {ISLAND_HEIGHT}</size></box></geometry>
</collision>''')
```

Two nested loops: 4 Y positions times 2 X positions equals 8 legs.

`links` starts as an empty list. `append()` adds an item to the end. This is the standard pattern for building up a collection of generated strings.

The Z coordinate is `ISLAND_HEIGHT/2 = 1.0`. A 2 m tall box centred at `Z=1.0` has its base at `Z=0`, sitting on the floor.

`<material>` colours the surface. The four numbers are red, green, blue and alpha (opacity), each from 0 to 1. `ambient` is the colour in shadow, `diffuse` the colour under direct light.

The `name` attributes must be unique within the model, which is why the loop variables are embedded in them.

3.13 Generating the Shelf Plates

```
for z in LEVELS_Z:
    links.append(f'''
<visual name="plate_{z}_visual">
    <pose>0 0 {z} 0 0 0</pose>
    <geometry><box><size>{ISLAND_DEPTH} {ISLAND_LENGTH} {PLATE_THICKNESS}</size></box></geometry>
    ...
</visual>
...''')
```

Three horizontal boards. Each is centred at X=0 and Y=0, spanning the full depth and length of the island, with only the Z coordinate varying.

Size order is always X, Y, Z: 1.6 deep, 12.0 long, 0.04 thick.

3.14 Assembling and Writing

```
links_str = ''.join(links)
```

`join()` concatenates list elements into a single string. The string it is called on is the separator; `''` means no separator, which is correct here because each generated fragment already begins with a newline.

```
sdf = f'''<?xml version="1.0" encoding="UTF-8"?>
<sdf version="1.9">
    <model name="{model_name}">
        <static>true</static>
        <link name="body">
{links_str}
        </link>
    </model>
</sdf>'''

with open(os.path.join(model_dir, 'model.sdf'), 'w') as f:
    f.write(sdf)
```

`with open(...) as f:` opens the file and guarantees it is closed afterwards, even if an error occurs. `'w'` is write mode, which overwrites any existing content.

3.15 What This File Produces

```
Input:  constants defining geometry
Process: nested loops generating XML fragments
Output: three folders, each with model.config and model.sdf
Result: Gazebo now recognises island_1, island_2 and island_3
```

4. `build_boxes.py`

4.1 Purpose

Generates 54 box models, each carrying a unique QR code, and writes a ground-truth inventory file recording where every box was placed. That file is later used to check how many boxes the drone actually found.

4.2 Imports

```
import os, qrcode, json
```

`qrcode` is a third-party library that generates QR code images. Installed with `pip install qrcode`.

`json` handles JavaScript Object Notation, a text format for structured data. Part of the standard library.

4.3 Constants

```
BOX_SIZE = 0.3
PLATE_THICKNESS = 0.04
LEVELS_Z = [0.35, 1.0, 1.65]
ISLAND_DEPTH = 1.6
islands_x = [-6.2, -1.6, 3.0]
box_y_positions = [-4.0, 0.0, 4.0]
```

`box_y_positions` places three boxes along the 12 m length of each island, spaced 4 m apart, well inside the ends.

4.4 The Nested Loop Structure

```
inventory = []
box_counter = 0

for island_i, cx in enumerate(islands_x):
    for level_i, plate_z in enumerate(LEVELS_Z):
        box_center_z = plate_z + PLATE_THICKNESS/2 + BOX_SIZE/2
        for y in box_y_positions:
            for face in ['sol', 'sag']:
                box_counter += 1
```

Four nested loops:

```
3 islands x 3 levels x 3 Y positions x 2 faces = 54 boxes
```

`box_counter += 1` is shorthand for `box_counter = box_counter + 1`.

4.5 Sitting the Box on the Shelf

```
box_center_z = plate_z + PLATE_THICKNESS/2 + BOX_SIZE/2
```

Worked through for the first level:

```
plate centre          = 0.35
plate half-thickness= 0.02  -> plate top surface at 0.37
box half-height       = 0.15  -> box centre at 0.52
```

Again the "position by centre" rule. Setting the box centre equal to the plate centre would bury half the box inside the shelf.

4.6 Generating the Identifier

```
box_id = f"KUTU-A{island_i+1}-K{level_i+1}-{face.upper()}-{box_counter:03d}"
```

Produces for example `KUTU-A1-K1-SOL-001`.

`{box_counter:03d}` is **format specification**: `d` means decimal integer, `03` means pad with zeros to three digits. So 1 becomes `001`. This keeps filenames sorting correctly.

`face.upper()` converts the string to uppercase.

Encoding the position into the identifier is deliberate. When the drone reports `KUTU-A2-K3-SAG-036`, the location is readable directly from the code, which made debugging far easier.

4.7 Creating the QR Image

```
qr = qrcode.QRCode(box_size=10, border=1)
qr.add_data(box_id)
qr.make(fit=True)
img = qr.make_image(fill_color='black', back_color='white')
img = img.convert('RGB').resize((300, 300))
```

`box_size` is pixels per QR module (the small squares). `border` is the quiet zone width in modules.

`qr.make(fit=True)` chooses the smallest QR version that fits the data.

`convert('RGB')` is required because the generator returns a 1-bit image, which Gazebo cannot use as a texture.

4.8 The Unique Filename Fix

```
qr_filename = f'qr_{box_counter:03d}.png'
qr_path = os.path.join(model_dir, qr_filename)
img.save(qr_path)
```

This is the single most important line in the file, and it was added only after a long debugging session.

Originally every box saved its texture as `qr.png`. Gazebo caches textures by filename. Because all 54 files had the same name, the renderer loaded the first one and reused it for every box. Every box in the warehouse displayed the same QR code.

The symptom was a detection rate of 1 out of 54, with the same identifier reported from every position. The cause was invisible in the flight logs and only became apparent when the saved camera frames were examined offline and every one showed the same code.

Giving each texture a unique filename raised detection from 1 to 43.

The general lesson: when detection performance is poor, verify the input data before tuning the detection algorithm.

4.9 Orienting the QR Face

```
if face == 'sol':
    qr_local_x = -BOX_SIZE/2 - 0.002
    pitch = -1.5708
else:
    qr_local_x = BOX_SIZE/2 + 0.002
    pitch = 1.5708
```

The QR is a flat plane. By default a plane in SDF lies horizontally with its normal pointing up along Z. To make it face sideways it must be rotated 90 degrees, which is 1.5708 radians.

The sign of the rotation determines which way the visible side points. Getting this wrong makes the texture face into the box, so it is invisible from outside. Both signs had to be tested to find the correct one.

The 0.002 offset lifts the plane 2 mm clear of the box surface. Without it the two surfaces occupy exactly the same space and the renderer cannot decide which to draw, producing a flickering artifact called **z-fighting**.

4.10 The Combined Box Model

```
sdf = f'''<?xml version="1.0" encoding="UTF-8"?>
<sdf version="1.9">
  <model name="{model_name}">
    <static>true</static>
    <link name="link">
      <visual name="box_visual">
        <geometry><box><size>{BOX_SIZE} {BOX_SIZE} {BOX_SIZE}</size></box></geometry>
        <material>...</material>
      </visual>
      <collision name="box_collision">
        <geometry><box><size>{BOX_SIZE} {BOX_SIZE} {BOX_SIZE}</size></box></geometry>
      </collision>
      <visual name="qr_visual">
        <pose>{qr_local_x} 0 0 0 {pitch} 0</pose>
        <geometry><plane><normal>0 0 1</normal><size>{BOX_SIZE*0.85} {BOX_SIZE*0.85}</size></plane>
        <material>
          <diffuse>1 1 1 1</diffuse>
          <pbr><metal><albedo_map>model://{model_name}/{qr_filename}</albedo_map></metal></pbr>
        </material>
      </visual>
    </link>
  </model>
</sdf>'''
```

One link containing three visuals: the box body, its collision shape, and the QR plane mounted on one face.

`<albedo_map>` applies an image as a surface texture. The `model://` prefix tells Gazebo to resolve the path relative to the model search directories.

`BOX_SIZE*0.85` makes the QR slightly smaller than the face so it does not overhang the edges.

4.11 Recording Ground Truth

```
box_edge_offset = ISLAND_DEPTH/2 - BOX_SIZE/2 - 0.02
x_offset = -box_edge_offset if face == 'sol' else box_edge_offset
world_x = cx + x_offset

inventory.append({
    "id": box_id, "model": model_name,
    "x": world_x, "y": y, "z": box_center_z,
    "face": face, "island": island_i+1, "level": level_i+1
})
```

A `if condition else B` is a **conditional expression**, a compact way to choose between two values.

The offset positions the box near the outer edge of the shelf so the camera can see it clearly:

```
0.8 (half depth) - 0.15 (half box) - 0.02 (clearance) = 0.63
```

The dictionary appended to `inventory` is the ground truth: where each box genuinely is. Comparing the drone's findings against this file is how detection accuracy is measured.

4.12 Writing the Ground Truth File

```
with open(os.path.expanduser('~/.autonomous_landing/inventory_ground_truth.json'), 'w') as f:
    json.dump(inventory, f, indent=2, ensure_ascii=False)
```

`json.dump()` writes a Python object to a file as JSON. `indent=2` formats it readably. `ensure_ascii=False` preserves non-ASCII characters rather than escaping them.

5. `build_scanner_drone.py`

5.1 Purpose

Builds the `x500_scanner` vehicle: PX4's standard x500 quadrotor with two extra cameras mounted facing left and right.

5.2 Model Composition

```
sdf = f'''<sdf version='1.9'>
  <model name='{model_name}'>
    <self_collide>false</self_collide>
    <include merge='true'>
      <uri>x500</uri>
    </include>
    {left_cam}
    {right_cam}
  </model>
</sdf>'''
```

`<include merge='true'>` pulls another model's contents into this one, rather than nesting it as a child. This is inheritance: everything the x500 has (motors, IMU, barometer, magnetometer, optical flow) is inherited, and the cameras are added on top.

`<self_collide>false</self_collide>` stops the physics engine testing parts of the same model against each other, which is wasted computation here.

Important history: the model originally inherited from `x500_flow` rather than `x500`. That model turned out to be broken in this PX4 version: EKF2 reported attitude: 0 and never produced a position estimate, so the vehicle could not arm. The fault was isolated by testing the stock `x500_flow` on its own and seeing it fail identically, which proved the problem was inherited rather than introduced. Switching the base to `x500` fixed it immediately, and optical flow was still available because `x500` already provides it.

5.3 The Camera Block

```
def camera_block(link_name, joint_name, y_offset, yaw):
    return f'''
    <joint name="{joint_name}" type="fixed">
      <parent>base_link</parent>
      <child>{link_name}</child>
      <pose relative_to="base_link">0.05 {y_offset} -0.02 0 0 {yaw}</pose>
    </joint>
    <link name="{link_name}">
      <pose relative_to="{joint_name}">0 0 0 0 0 0</pose>
      <inertial>
        <mass>0.01</mass>
        <inertia>
          <ixx>0.00001</ixx><iyy>0.00001</iyy><izz>0.00001</izz>
          <ixy>0</ixy><ixz>0</ixz><iyz>0</iyz>
        </inertia>
      </inertial>
      <sensor name="camera" type="camera">
        <gz_frame_id>{link_name}</gz_frame_id>
        <always_on>1</always_on>
        <update_rate>30</update_rate>
        <camera name="camera">
          <horizontal_fov>1.0472</horizontal_fov>
          <image>
            <width>640</width>
            <height>480</height>
            <format>R8G8B8</format>
          </image>
          <clip><near>0.1</near><far>100</far></clip>
        </camera>
      </sensor>
    </link>'''
```

Defining a function avoids writing the same twenty lines twice with two numbers changed.

`<joint type="fixed">` rigidly attaches the camera link to the airframe. A fixed joint permits no relative motion; it exists purely to define the attachment and its offset.

`<inertial>` gives the link mass and rotational inertia. The physics engine requires these on every link. The values are deliberately tiny so the cameras do not measurably alter the flight dynamics.

`<horizontal_fov>1.0472</horizontal_fov>` is 60 degrees in radians. At the 1.67 m aisle standoff used in this project this covers roughly 1.9 m horizontally, comfortably more than one box.

`<clip>` sets the near and far rendering limits. Objects closer than 0.1 m or further than 100 m are not drawn.

5.4 Camera Placement

```
left_cam = camera_block("camera_left_link", "camera_left_joint", 0.12, 1.5708)
right_cam = camera_block("camera_right_link", "camera_right_joint", -0.12, -1.5708)
```

The left camera is offset +0.12 m in Y (left, in the vehicle frame) and rotated +90 degrees in yaw so it looks left. The right camera mirrors this.

This arrangement is what allows a single pass down an aisle to scan the shelf faces on both sides at once, halving the number of passes needed.

6. `build_v2_world.py`

6.1 Purpose

Assembles all the previously generated models into a single Gazebo world file, and writes the ArUco marker position map.

6.2 Reading the Ground Truth

```
with open(INV_PATH) as f:
    inventory = json.load(f)
```

`json.load()` parses a JSON file into Python objects. This is the counterpart of `json.dump()`. The box positions are read back from the file that `build_boxes.py` wrote, so the two scripts cannot disagree about where a box is.

6.3 The Include Helper

```
includes = []
counter = {}

def add_include(model_type, x, y, z, roll=0, pitch=0, yaw=0):
    counter[model_type] = counter.get(model_type, 0) + 1
    unique_name = f"{model_type}_{counter[model_type]}"
    includes.append(f'''    <include>
        <uri>model://{model_type}</uri>
        <name>{unique_name}</name>
        <pose>{x} {y} {z} {roll} {pitch} {yaw}</pose>
    </include>''')
```

`counter` is a **dictionary**: a mapping from keys to values, written with braces. Here it counts how many instances of each model type have been placed.

`counter.get(key, 0)` returns the value for the key, or 0 if the key is not present. Without the default, looking up a missing key raises an error.

`roll=0, pitch=0, yaw=0` are **default parameter values**. If the caller omits them the defaults are used, so most calls only pass position.

The `<name>` tag is essential. Gazebo requires every instance in a world to have a unique name. Placing three islands with the same name produces:

```
Error Code 2: model with name[warehouse_shelf] already exists
```

and the world fails to load entirely. This error was encountered during development and is the reason the counter exists.

6.4 Placing the Static Structure

```
add_include('warehouse_wall', 0, 10, 2, 0, 0, 0)
add_include('warehouse_wall', 0, -10, 2, 0, 0, 0)
add_include('warehouse_wall', 10, 0, 2, 0, 0, 1.5708)
add_include('warehouse_wall', -10, 0, 2, 0, 0, 1.5708)
```

Four walls forming a 20 x 20 m enclosure. The north and south walls need no rotation; the east and west walls are rotated 90 degrees in yaw.

Z is 2 because the wall is 4 m tall and positioned by its centre.

```
islands_x = [-6.2, -1.6, 3.0]
for i, cx in enumerate(islands_x):
    add_include(f'island_{i+1}', cx, 0, 0, 0, 0, 0)
```

Z is 0 here, not 1.0. The island model already describes itself extending from 0 to 2 m, so adding a Z offset would lift it into the air. An earlier version of this file used Z=1.0 and produced exactly that: shelves floating one metre above the floor.

6.5 Placing the Boxes

```
for item in inventory:
    add_include(item['model'], item['x'], item['y'], item['z'], 0, 0, 0)
```

Every box is placed at the coordinates recorded in the ground truth file. Because both the placement and the validation read from the same source, they cannot drift apart.

6.6 The Marker Map

```
corridor_x_centers = [-8.5, -3.9, 0.7, 5.3]
marker_id = 1
marker_map = {}
for cxc in corridor_x_centers:
    for y_end in [-8.0, 8.0]:
        marker_model = f"corridor_marker_{marker_id}"
        add_include(marker_model, cxc, y_end, 0, 0, 0, 0)
        marker_map[str(marker_id)] = {"x": cxc, "y": y_end}
        marker_id += 1
```

Eight markers, one at each end of each aisle, each with a **unique ID**.

An earlier version reused six IDs across eight positions using a modulo operation. That made the map ambiguous: seeing marker 1 could mean the vehicle was at either of two positions, which defeats the entire purpose of using markers for position correction. Every marker must be unique for the correction to be well defined.

The map is written to JSON so the flight code can look up, for any detected ID, exactly where that marker is:

```
{ "1": {"x": -8.5, "y": -8.0}, "2": {"x": -8.5, "y": 8.0}, ... }
```

6.7 The World Header

```
<physics type="ode">
  <max_step_size>0.004</max_step_size>
  <real_time_factor>1.0</real_time_factor>
  <real_time_update_rate>250</real_time_update_rate>
</physics>
```

ODE is the physics engine. `max_step_size` 0.004 s and update rate 250 Hz together mean the simulation advances in 4 ms increments, 250 times per second, so simulated time matches wall-clock time.

```
<light name="sun" type="directional">
```

A directional light models sunlight: parallel rays from a fixed direction. A second `point` light is added at ceiling height to represent indoor lighting.

Relevant to the QR problem: these lighting settings determine how bright the white areas of the QR textures appear to the camera. The rendered white came out as mid grey, which is why the raw camera frames could not be decoded and why the crop-then-threshold pipeline was needed.

```
<spherical_coordinates>
  <surface_model>EARTH_WGS84</surface_model>
  <world_frame_orientation>ENU</world_frame_orientation>
  <latitude_deg>47.397971057728974</latitude_deg>
  ...
</spherical_coordinates>
```

Defines where on Earth the world sits. PX4 needs this even with GPS disabled, because it uses the location to model the magnetic field for the magnetometer.

7. `4022_gz_x500_scanner` — The PX4 Airframe

7.1 Purpose

Tells PX4 which vehicle model to spawn and which parameters to apply. Without this file the custom vehicle cannot be launched through the normal build system.

7.2 Contents

```
#!/bin/sh
#
# @name Gazebo x500 scanner (left/right cameras + optical flow)
#
# @type Quadrotor
#
PX4_SIM_MODEL=${PX4_SIM_MODEL:=x500_scanner}
. ${R}etc/init.d-posix/airframes/4001_gz_x500

echo "Disabling simulated GPS - indoor optical flow operation"
param set-default SYS_HAS_GPS 0
param set-default SIM_GPS_USED 0
param set-default EKF2_GPS_CTRL 0
```

7.3 Line by Line

```
#!/bin/sh
```

The shebang, telling the system to execute this with the shell interpreter.

```
# @name ...
# @type Quadrotor
```

These comments are parsed by PX4's build system to generate documentation. The format matters even though they look like ordinary comments.

```
PX4_SIM_MODEL=${PX4_SIM_MODEL:=x500_scanner}
```

Shell syntax meaning: if `PX4_SIM_MODEL` is already set, keep it; otherwise set it to `x500_scanner`. This allows the value to be overridden from outside while providing a sensible default.

This variable name is the one that matters. During development `PX4_GZ_MODEL` was set instead and had no effect. The spawn script reads `PX4_SIM_MODEL` and strips the `gz_` prefix from it:

```
MODEL_NAME="${PX4_SIM_MODEL#*gz_}"
```

Reading the actual spawn script was what resolved this. Guessing environment variable names is unproductive; the source is authoritative.

```
. ${R}etc/init.d-posix/airframes/4001_gz_x500
```

The leading dot is the shell **source** command: execute that file in the current shell so its settings apply here. This inherits the entire standard x500 configuration (motor layout, control allocation, hover thrust) rather than repeating it.

`${R}` is a variable PX4 sets to the root of its config tree.

```
param set-default SYS_HAS_GPS 0
param set-default SIM_GPS_USED 0
param set-default EKF2_GPS_CTRL 0
```

Three separate settings are needed because they act at different levels:

Parameter	Effect
<code>SYS_HAS_GPS</code>	The vehicle has no GPS receiver fitted
<code>SIM_GPS_USED</code>	The simulator should not publish GPS data
<code>EKF2_GPS_CTRL</code>	The state estimator must not fuse GPS even if present

With these set, EKF2 falls back to optical flow and range for position.

7.4 File Numbering and Registration

The filename `4022_gz_x500_scanner` follows PX4's convention:

```
4022      airframe ID, must be unique
gz        Gazebo simulation
x500_scanner  model name
```

The build system finds airframes with a glob pattern:

```
file(GLOB gz_airframes ${PX4_SOURCE_DIR}/ROMFS/.../airframes/*_gz_*)
```

and then generates one build target per airframe and world:

```
add_custom_target(gz_${model_name}_${world_name} ...)
```

So placing the file in that directory and re-running CMake automatically creates the target `gz_x500_scanner_warehouse_v2`. No manual registration is needed, which is why re-running `cmake .` in the build directory was sufficient.

7.5 Installation

```
cp 4022_gz_x500_scanner ~/PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes/  
chmod +x ~/PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes/4022_gz_x500_scanner  
cd ~/PX4-Autopilot/build/px4_sitl_default && cmake .
```

`chmod +x` marks the file executable, which is required because PX4 runs it as a script.

Do not use `sudo` for this. Doing so makes the file owned by root, after which `chmod` fails with a permission error. This happened during development and had to be undone with `chown`.

8. `flow_flight_test.py` — GPS-Free Flight Test

8.1 Purpose

Before building anything on top of GPS-free flight, verify that it works at all: can the vehicle arm, take off, hold position, and how much does it drift?

8.2 Async Programming Basics

```
import asyncio
```

MAVSDK is asynchronous. Three keywords matter:

`async def` defines a coroutine, a function that can pause and resume:

```
async def run():  
    ...
```

`await` pauses the coroutine until an operation finishes, letting other code run meanwhile:

```
await drone.action.arm()
```

`asyncio.run()` starts the event loop and runs a coroutine to completion:

```
asyncio.run(run())
```

Why this matters here: the drone must be commanded while camera frames arrive and telemetry streams update. Sequential code would block on one and starve the others.

8.3 Connecting

```
drone = System()
await drone.connect(system_address="udp://:14540")
```

`System()` creates a vehicle handle. `connect()` opens a MAVLink connection. Port 14540 is PX4's default offboard API port.

```
async for state in drone.core.connection_state():
    if state.is_connected:
        break
```

`async for` iterates an asynchronous stream, waiting for each item. Telemetry in MAVSDK is delivered as such streams rather than as single calls.

`break` exits as soon as the connection is up.

8.4 The Health Check

```
async for h in drone.telemetry.health():
    print(f" gyro={h.is_gyrometer_calibration_ok} ... "
          f"local_pos={h.is_local_position_ok} home={h.is_home_position_ok}")
    if h.is_local_position_ok and h.is_home_position_ok:
        sensors_ready = True
        break
```

Two flags matter for GPS-free flight:

- `is_local_position_ok` — EKF2 has a usable local position estimate
- `is_home_position_ok` — a home reference has been established

PX4 refuses to arm without these. During development both stayed false and arming failed with `COMMAND_DENIED`. Printing every flag individually was what made the failure diagnosable: it showed the IMU and magnetometer were fine while position was not, narrowing the fault to the estimator rather than the sensors.

8.5 Arming and Takeoff

```
await drone.action.arm()
await drone.action.takeoff()
```

`arm()` spins up the motors. `takeoff()` climbs to a default altitude using PX4's internal takeoff logic, not offboard control.

8.6 Measuring Drift

```
await drone.offboard.set_velocity_body(VelocityBodyYawspeed(0.0, 0.0, 0.0, 0.0))
await drone.offboard.start()

positions = []
for i in range(20):
    async for odom in drone.telemetry.position_velocity_ned():
        positions.append((odom.position.north_m, odom.position.east_m))
        break
    await asyncio.sleep(0.5)

start_n, start_e = positions[0]
end_n, end_e = positions[-1]
drift = ((end_n - start_n)**2 + (end_e - start_e)**2) ** 0.5
```

Offboard mode must receive a setpoint before it can be started. Calling `start()` without first sending a setpoint is rejected. This is a safety rule: PX4 will not hand over control until it knows what the controller wants.

The zero-velocity setpoint commands the vehicle to hold still.

`positions[-1]` indexes from the end; -1 is the last element.

`** 0.5` raises to the power of one half, which is the square root. The expression is the Euclidean distance between the first and last position.

Measured result: approximately 0.05 m of drift over 10 seconds, which established that optical flow was accurate enough to proceed.

9. `waypoint_nav.py` — Navigation Test

9.1 Purpose

Verify that the vehicle can follow a planned route, with no camera processing involved. Separating navigation from perception means a later failure can be attributed to one or the other.

9.2 Building the Route

```
def build_route():
    route = []
    going_north = True
    for c_idx in range(CORRIDORS_TO_SCAN):
        x = CORRIDOR_X[c_idx]
        levels = FLIGHT_Z if c_idx % 2 == 0 else list(reversed(FLIGHT_Z))
        for z in levels:
            if going_north:
                route.append((x, Y_SOUTH, z, 0.0))
                route.append((x, Y_NORTH, z, 0.0))
            else:
                route.append((x, Y_NORTH, z, 0.0))
                route.append((x, Y_SOUTH, z, 0.0))
            going_north = not going_north
    return route
```

This generates the boustrophedon pattern.

`c_idx % 2 == 0` uses the **modulo** operator, which gives the remainder of a division. It is zero for even numbers, so this alternates between corridors.

`list(reversed(...))` reverses the level order. Combined with the alternating `going_north` flag, this means the vehicle never has to fly an empty leg back to the start of a corridor.

`going_north = not going_north` flips a boolean.

The resulting pattern:

```
corridor 1, level 1: south -> north
corridor 1, level 2: north -> south
corridor 1, level 3: south -> north
corridor 2, level 3: north -> south
corridor 2, level 2: south -> north
...
```

9.3 The Coordinate Conversion

```
SPAWN_X, SPAWN_Y = -8.5, -9.0
```

```
target_n = y - SPAWN_Y
target_e = x - SPAWN_X
target_d = -z
```

This is the conversion between Gazebo world coordinates and PX4 NED coordinates, and it is the single most error-prone part of the project.

Three things are happening at once:

Axis renaming. Gazebo Y corresponds to NED North; Gazebo X corresponds to NED East. The vehicle is spawned with a 90 degree yaw, which is what produces this correspondence.

Origin shift. NED coordinates are relative to where the vehicle started, not to the world origin. Subtracting the spawn position converts between the two frames.

Sign inversion. NED's third axis points **down**, so an altitude of 1.8 m becomes a D coordinate of -1.8.

A worked example:

```
Gazebo target:  x = -3.9, y = 8.0, z = 1.15
Spawn:          x = -8.5, y = -9.0

North = 8.0 - (-9.0) = 17.0
East  = -3.9 - (-8.5) = 4.6
Down  = -1.15
```

A trap encountered during development: `position_velocity_ned` returns position relative to the home point, which is the spawn location. Checking it right after spawn always reads approximately (0, 0) regardless of where in the world the vehicle actually is. This made it look as though `PX4_GZ_MODEL_POSE` was being ignored when in fact it was working. The world position had to be read from Gazebo directly:

```
gz model -m x500_scanner_0 -p
```

which reported $x=-8.0$, $y=-4.0$, confirming the spawn had worked.

9.4 Tracking Position

```
current_pos = {"n": 0.0, "e": 0.0, "d": 0.0}

async def track_position(drone):
    global current_pos
    async for odom in drone.telemetry.position_velocity_ned():
        current_pos["n"] = odom.position.north_m
        current_pos["e"] = odom.position.east_m
        current_pos["d"] = odom.position.down_m
```

`global` declares that the function modifies the module-level variable rather than creating a local one.

```
asyncio.create_task(track_position(drone))
```

`create_task()` schedules a coroutine to run concurrently in the background. It is not awaited, so execution continues immediately while this keeps updating `current_pos` for the rest of the flight.

9.5 Waypoint Arrival

```
def distance_to(target_n, target_e, target_d):
    dn = target_n - current_pos["n"]
    de = target_e - current_pos["e"]
    dd = target_d - current_pos["d"]
    return math.sqrt(dn*dn + de*de + dd*dd)
```

Euclidean distance in three dimensions.

```
while elapsed < TIMEOUT_PER_WP:
    await drone.offboard.set_position_ned(
        PositionNedYaw(target_n, target_e, target_d, yaw))
    if distance_to(target_n, target_e, target_d) < WAYPOINT_TOLERANCE:
        return True
    await asyncio.sleep(step)
    elapsed += step
```

The setpoint must be **resent continuously**. PX4 has an offboard timeout of roughly 500 ms; if setpoints stop arriving it drops out of offboard mode as a safety measure. Sending once and waiting would fail.

The tolerance of 0.4 m accounts for the fact that a real controller never settles exactly on a setpoint.

The timeout prevents an unreachable waypoint from stalling the mission indefinitely.

9.6 Result

All 12 waypoints were reached with a mean error of about 0.3 m, confirming that planned navigation using optical flow was viable before any camera code was written.

10. `warehouse_scanner.py` — The Main System

This file combines navigation and perception. It is the production script.

10.1 Imports

```
import os
os.environ["PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION"] = "python"
```

This must appear **before** importing the Gazebo transport library. `os.environ` is a dictionary of environment variables. Setting this one forces protobuf to use its pure-Python implementation, avoiding a version conflict between the compiled protobuf bundled with Gazebo and the one Python has. Placing it after the import has no effect, because the setting is read at import time.

```
import gz.transport13 as trans
from gz.msgs10.image_pb2 import Image
```

`gz.transport13` is Gazebo's messaging library. `as trans` gives it a shorter local name.

`from X import Y` imports one specific name rather than the whole module. `Image` is the message type describing a camera frame.

10.2 Configuration Block

```
CORRIDOR_X = [-8.5, -3.9, 0.7, 5.3]
FLIGHT_Z = [0.5, 1.15, 1.8]
Y_SOUTH, Y_NORTH = -8.0, 8.0
SPAWN_X, SPAWN_Y = -8.5, -9.0

WAYPOINT_TOLERANCE = 0.4
CRUISE_SPEED = 0.7
TIMEOUT_MARGIN = 15.0
```

`FLIGHT_Z` is each shelf level plus 0.15 m. The camera sits slightly above the shelf surface so the box face fills more of the frame.

`CORRIDOR_X` originally ended in 6.9. That put the vehicle 3.27 m from the shelf in the outermost aisle, against 1.67 m elsewhere, and at that range the QR codes were too small in the image to decode. Every missed detection in one test run came from that aisle. Moving the centre line to 5.3 restored the standoff to 1.67 m.

10.3 The QR Decoding Pipeline

This is the most involved function in the project.

```

def decode_qr(frame):
    results = []
    try:
        ok, points = qr_detector.detectMulti(frame)
        if not ok or points is None:
            ok_single, points_single = qr_detector.detect(frame)
            if not ok_single or points_single is None:
                return results
            points = points_single

        for quad in points:
            p = quad.astype(int)
            x1 = max(0, p[:, 0].min() - 8)
            y1 = max(0, p[:, 1].min() - 8)
            x2 = min(frame.shape[1], p[:, 0].max() + 8)
            y2 = min(frame.shape[0], p[:, 1].max() + 8)
            crop = frame[y1:y2, x1:x2]
            if crop.size == 0:
                continue

            gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY)
            _, binary = cv2.threshold(gray, 0, 255,
                                      cv2.THRESH_BINARY + cv2.THRESH_OTSU)

            data, _, _ = qr_detector.detectAndDecode(binary)
            if data:
                results.append(data)
                continue

            upscaled = cv2.resize(binary, None, fx=3.0, fy=3.0,
                                   interpolation=cv2.INTER_CUBIC)
            data_up, _, _ = qr_detector.detectAndDecode(upscaled)
            if data_up:
                results.append(data_up)
    except Exception:
        pass
    return results

```

Why the naive approach fails

`qr_detector.detectAndDecode(frame)` on the raw image locates the code but returns an empty string. The reason is that Gazebo renders the white quiet zone as mid grey. Measured on a saved frame:

```
min = 40, max = 200, mean = 130
```

A QR decoder needs near-black against near-white. This contrast is too low.

Why global thresholding also fails

Applying Otsu thresholding to the whole frame was the obvious fix and did not work either. Otsu computes a **single** threshold for the entire image. A camera frame contains shelves, floor, walls and boxes at different brightness levels, and one threshold cannot separate all of them correctly. The QR region ends up on the wrong side of it.

Why crop-then-threshold works

Restricting the threshold computation to the QR region alone means Otsu only has to separate two populations, the dark and light modules of the code itself. Within that small area the illumination is uniform and the separation is clean.

Verified experimentally on a saved frame:

```
raw frame, detectAndDecode      -> ''
whole-frame Otsu                 -> ''
crop then Otsu                   -> 'KUTU-A1-K1-S0L-001'
```

Line notes

`detectMulti()` finds several codes in one frame; `detect()` finds one. The fallback covers cases where the multi-detector fails but the single one does not.

`p[:, 0]` is NumPy **slice notation**: all rows, column 0. The four corner points are stored as rows, so this extracts all the x coordinates.

`max(0, ... - 8)` and `min(frame.shape[1], ... + 8)` add an 8 pixel margin while clamping to the image bounds, so the crop never runs off the edge.

`frame[y1:y2, x1:x2]` crops. NumPy indexes images as **[row, column]**, meaning Y comes first. Reversing these is a common bug.

`cv2.THRESH_BINARY + cv2.THRESH_OTSU` combines two flags. Otsu computes the threshold automatically, which is why the threshold argument is passed as 0.

`INTER_CUBIC` is the interpolation method used when upscaling; it produces smoother edges than nearest-neighbour, which helps the decoder on small codes.

`except Exception: pass` swallows errors. This is normally poor practice, but this function runs inside a camera callback and an exception there would kill the stream, ending the mission. A dropped frame is preferable.

10.4 Camera Callbacks

```
def on_image_left(msg):
    global detected_left
    try:
        img = np.frombuffer(msg.data, dtype=np.uint8).reshape(
            (msg.height, msg.width, 3))
        detected_left = decode_qr(cv2.cvtColor(img, cv2.COLOR_RGB2BGR))
    except Exception:
        pass
```

`np.frombuffer()` interprets raw bytes as a NumPy array without copying them.

`.reshape((height, width, 3))` gives the flat byte array its image shape: rows, columns, and three colour channels.

`cv2.cvtColor(img, cv2.COLOR_RGB2BGR)` converts colour ordering. Gazebo publishes RGB; OpenCV expects BGR. Omitting this swaps red and blue, which happens not to affect QR decoding but would corrupt any colour-based detection.

10.5 Recording a Detection

```
def record_detection(qr_id, side):
    if qr_id in inventory:
        return False

    gx, gy, gz = ned_to_gazebo(current_pos["n"], current_pos["e"],
                               current_pos["d"])
    lateral_offset = -0.8 if side == "left" else 0.8

    inventory[qr_id] = {
        "id": qr_id,
        "detected_by": side,
        "estimated_x": round(gx + lateral_offset, 2),
        ...
    }
```

`inventory` is a dictionary keyed by QR content. The membership test at the top means each code is recorded once no matter how many frames it appears in.

`ned_to_gazebo()` reverses the coordinate conversion so the output is in world coordinates, which is what a warehouse operator would expect.

The 0.8 m lateral offset is an approximation: the box is assumed to be that far to the side of the vehicle, on whichever side detected it. A more accurate estimate would use the code's pixel position and the camera intrinsics to triangulate, which is listed as future work.

10.6 Carrot Following

```
dt = 0.1
travelled = 0.0
elapsed = 0.0
max_time = leg_length / CRUISE_SPEED + TIMEOUT_MARGIN

while elapsed < max_time:
    travelled = min(leg_length, travelled + CRUISE_SPEED * dt)
    fraction = 1.0 if leg_length == 0 else travelled / leg_length
    carrot_n = start_n + (target_n - start_n) * fraction
    carrot_e = start_e + (target_e - start_e) * fraction
    carrot_d = start_d + (target_d - start_d) * fraction

    await drone.offboard.set_position_ned(
        PositionNedYaw(carrot_n, carrot_e, carrot_d, 0.0))
    await poll_cameras()

    if (travelled >= leg_length
        and distance_to(target_n, target_e, target_d) < WAYPOINT_TOLERANCE):
        return True

    await asyncio.sleep(dt)
    elapsed += dt
```

The problem this solves

The previous implementation divided each leg into fixed points and waited for arrival at each one before issuing the next. The vehicle accelerated towards a point, decelerated on arrival, then accelerated again, producing a visible lurching motion. The oscillation also disturbed the camera view.

How it works

The setpoint is advanced on a **timer** rather than on arrival:

```
travelled = travelled + CRUISE_SPEED * dt
```

With `CRUISE_SPEED` 0.7 m/s and `dt` 0.1 s, the target moves 7 cm every cycle. The vehicle continuously chases a point that is always slightly ahead of it, which produces smooth motion at roughly constant speed. The technique is known as carrot following.

`min(leg_length, ...)` stops the carrot at the waypoint rather than overshooting.

`fraction` converts distance travelled into a position along the line by linear interpolation.

The `leg_length == 0` guard avoids a division by zero when a waypoint coincides with the current position.

Effect

Waypoints reached went from 23/24 to 24/24 and codes decoded from 51 to 52, while the motion became visibly smooth. Both improvements came from the same change: a steadier platform gives the camera a steadier view.

10.7 Writing the Output

```
with open(OUTPUT_JSON, "w", encoding="utf-8") as handle:
    json.dump({
        "scan_date": datetime.now().isoformat(timespec="seconds"),
        "localization": "optical flow (no GPS)",
        "total_detected": len(inventory),
        "waypoints_completed": f"{reached}/{len(route)}",
        "items": list(inventory.values()),
    }, handle, indent=2, ensure_ascii=False)
```

`datetime.now().isoformat()` produces a standard timestamp such as `2026-08-13T14:32:07`.

`inventory.values()` returns the dictionary values; `list()` converts that view into a list, which JSON can serialise.

11. Python Syntax Reference

Everything used in this project, in one place.

Variables and Types

```
x = 5                # int
y = 3.14             # float
name = "text"        # str
flag = True           # bool
items = [1, 2, 3]     # list, ordered, mutable
pair = (1, 2)         # tuple, ordered, immutable
table = {"a": 1}      # dict, key-value mapping
```

Lists

```
items[0]             # first element
items[-1]            # last element
items[1:3]           # slice, elements 1 and 2
len(items)           # length
items.append(4)       # add to end
list(reversed(items)) # reversed copy
```

Dictionaries

```
table["a"]           # look up, error if missing
table.get("a", 0)     # look up with default
table["b"] = 2        # insert or update
"a" in table          # membership test
table.values()        # all values
table.setdefault("c", []) # insert default if absent, then return
```

Strings

```
f"value is {x}"      # f-string interpolation
f"{x:03d}"           # zero-padded to 3 digits
f"{x:.2f}"           # 2 decimal places
'''multi
line'''              # triple-quoted string
"a,b".split(",")     # split into list
",".join(["a", "b"]) # join list into string
text.upper()         # uppercase
text.replace("a", "b") # substitution
```

Control Flow

```
if cond:
    ...
elif other:
    ...
else:
    ...

value = A if cond else B    # conditional expression

for item in items: ...
for i, item in enumerate(items): ...
for i in range(5): ...      # 0,1,2,3,4

while cond: ...

break        # exit loop
continue     # next iteration
```

Functions

```
def f(a, b=10):            # b has a default
    return a + b

f(1)                       # 11
f(1, 2)                    # 3
f(a=1, b=2)                # keyword arguments
```

Async

```
async def f(): ...         # coroutine
await f()                  # wait for completion
async for x in stream: ...
asyncio.create_task(f())   # run concurrently
asyncio.run(f())           # start event loop
await asyncio.sleep(0.1)   # non-blocking pause
```

Scope

```
counter = 0

def f():
    global counter    # modify the module-level variable
    counter += 1
```

Files

```
with open(path) as f:    # read, auto-closed
    content = f.read()

with open(path, "w") as f: # write, overwrites
    f.write(text)

json.load(f)             # JSON file -> Python object
json.dump(obj, f, indent=2) # Python object -> JSON file
```

Errors

```
try:
    risky()
except Exception as e:
    print(e)
except Exception:
    pass                # ignore, only when a failure is acceptable
```

Paths

```
os.path.expanduser("~/x")    # expand home directory
os.path.join(a, b)           # join path components
os.makedirs(p, exist_ok=True) # create directory
os.path.basename(p)          # filename only
```

NumPy

```
np.frombuffer(data, dtype=np.uint8)    # bytes -> array, no copy
arr.reshape((h, w, 3))                 # change shape
arr[:, 0]                              # all rows, column 0
arr[a:b, c:d]                          # 2D slice
arr.min(), arr.max(), arr.mean()       # statistics
arr.astype(int)                        # change element type
```

OpenCV

```
cv2.imread(path)                       # load image
cv2.imwrite(path, img)                  # save image
cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  # convert colour space
cv2.threshold(g, 0, 255,
              cv2.THRESH_BINARY + cv2.THRESH_OTSU) # automatic thresholding
cv2.resize(img, None, fx=3, fy=3,
           interpolation=cv2.INTER_CUBIC) # scale
img.shape                               # (height, width, channels)

det = cv2.QRCodeDetector()
det.detect(img)                         # locate one
det.detectMulti(img)                   # locate several
det.detectAndDecode(img)               # locate and read
```

MAVSDK

```
drone = System()
await drone.connect(system_address="udp://:14540")

async for s in drone.core.connection_state(): ...
async for h in drone.telemetry.health(): ...
async for p in drone.telemetry.position(): ...
async for o in drone.telemetry.position_velocity_ned(): ...

await drone.action.arm()
await drone.action.takeoff()
await drone.action.land()

await drone.offboard.set_position_ned(PositionNedYaw(n, e, d, yaw))
await drone.offboard.set_velocity_body(VelocityBodyYawspeed(vx, vy, vz, yawrate))
await drone.offboard.start()
await drone.offboard.stop()
```

Two rules that are easy to get wrong:

1. Send a setpoint **before** calling `offboard.start()` , or it is rejected.
2. Keep sending setpoints at least every 500 ms, or PX4 exits offboard mode.

12. Debugging Methodology

The techniques below solved the actual problems on this project and generalise beyond it.

12.1 Measure Rather Than Guess

When the vehicle appeared to drift sideways, the useful step was not to speculate about causes but to write a script that flew straight with no correction and logged ground-truth position every 0.5 s. That produced a number: 0.42 m over 9.5 s, accelerating. The measurement redirected the investigation immediately.

12.2 Bisect the System

When EKF2 failed to initialise, the fault was isolated by testing successively simpler configurations:

```
custom model + custom world -> fails
custom model + default world -> fails      (world is not the cause)
stock x500_flow + default      -> fails      (our changes are not the cause)
stock x500 + default           -> works      (x500_flow is the cause)
```

Four tests localised the fault precisely. Each test changed exactly one thing.

12.3 Save Intermediate Data

QR detection was failing in flight but the reason was not visible in the logs. Writing every camera frame to disk during the flight and analysing them offline afterwards revealed that all 54 boxes were showing the same code. That is not something a detection-rate number can tell you.

12.4 Read the Source

The `PX4_GZ_MODEL_POSE` and `PX4_SIM_MODEL` confusion was resolved by opening PX4's spawn script and reading which variable it actually uses:

```
grep -n "MODEL" ROMFS/px4fmu_common/init.d-posix/px4-rc.gzsim
```

Guessing environment variable names produced several wasted attempts. The source answered it in one command.

12.5 Verify Assumptions Before Tuning

The instinct on a 1-in-54 detection rate is to tune the detector. The actual fault was upstream, in the texture data. Checking that the input is what you believe it to be should come before adjusting the algorithm that consumes it.

12.6 Distinguish Symptom From Cause

Adding scan pauses seemed a plausible fix for missed detections and was implemented. It changed nothing, because the cause was the texture collision. Once that was fixed, the pauses turned out to be unnecessary and were removed.

A fix that does not change the outcome is evidence that the diagnosis is wrong.

Appendix A — Commands

```
# Regenerate everything
python3 build_islands.py
python3 build_boxes.py
python3 build_scanner_drone.py
python3 build_v2_world.py

# Install the airframe (once)
cp 4022_gz_x500_scanner ~/PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes/
chmod +x ~/PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes/4022_gz_x500_scanner
cd ~/PX4-Autopilot/build/px4_sitl_default && cmake .

# Launch the simulation
cd ~/PX4-Autopilot
export PX4_GZ_MODEL_POSE="-8.5,-9,0,0,0,1.5708"
make px4_sitl gz_x500_scanner_warehouse_v2

# Headless (faster, no window)
export HEADLESS=1

# Run the scan
python3 warehouse_scanner.py

# Clean up stuck processes
pkill -9 -f px4; pkill -9 -f gz; sleep 5
```

Appendix B — Diagnostic Commands

```
# List all Gazebo topics
gz topic -l

# Filter for a specific sensor
gz topic -l | grep -i camera

# Read one message from a topic
gz topic -e -t /world/warehouse_v2/... -n 1

# True world position of the vehicle
gz model -m x500_scanner_0 -p
```

Inside the PX4 console (pxh>):

```
ekf2 status          estimator state
sensors status       sensor selection and rates
listener vehicle_local_position  position estimate
listener sensor_optical_flow      raw flow measurements
listener distance_sensor           range measurements
```

Appendix C – Errors Encountered and Their Causes

Symptom	Cause	Fix
model with name[x] already exists	Duplicate instance names in the world	Add a <name> tag with a counter
Unable to find uri[model://x]	Model directory not on the search path	Set GZ_SIM_RESOURCE_PATH
Shelves floating above the floor	Z offset applied twice	Place at Z=0; the model already spans 0 to 2
QR texture invisible	Plane rotated so its face points inward	Flip the sign of the pitch rotation
Markers flickering	Two surfaces at identical depth (z-fighting)	Offset one by a few millimetres
Arming denied	EKF2 had no position estimate	Base the model on x500 , not x500_flow
Spawn pose apparently ignored	Wrong environment variable	Use PX4_SIM_MODEL , not PX4_GZ_MODEL
Position reads (0,0) after spawn	NED is relative to home, not the world	Query Gazebo for world position
1 of 54 codes read, all identical	Shared texture filename, cache collision	Unique filename per texture
One aisle detecting nothing	Vehicle 3.27 m from shelf instead of 1.67 m	Move the aisle centre line
Lurching flight	Tolerance-gated stepping	Carrot following
bind error: Address in use	A previous script still holds port 14540	pkill -9 -f warehouse_scanner